ServiceOps complete platform manual

Version 1.27.18 · CMDB enrichment edition

See `docs/MASTER_REFERENCE.md` for a single merged document that also includes the governed backlog, traceability matrix, ITIL hierarchy reference, BP-001 blueprint traceability, REST API reference, and a fresh whole-application audit alongside this manual — useful as one-stop context for an AI agent or new engineer.

1. Purpose and operating model

ServiceOps is an enterprise service-management platform for requesters, fulfillers, team managers, CCB members, and platform administrators. It includes incident, request, problem, change, catalog, knowledge, CMDB, asset, SLA, approval, audit, analytics, and enterprise workspaces.

No deployment mechanism can eliminate every infrastructure or operator failure. The ServiceOps production standard therefore uses prevention, validation, observability, tested recovery, least privilege, immutable releases, and documented rollback rather than claiming infallibility.

2. Roles and teams

Role · Normal responsibilities

Requester · Submit and track requests and incidents, search knowledge

Agent · Triage, assign, fulfill, document, and resolve operational work

Manager · Agent work plus team oversight and manager approvals

CCB member · Review planned changes, risk, evidence, schedule, and backout plan

Administrator · Identity, ITIL configuration, CMDB, audit, and platform operation

CoreApps, Database, Network, Windows, Unix, and SSD are standard fulfillment teams. Administrators can map AD groups such as `gg_unix` to the Unix team; membership reconciles whenever the user logs in through AD/LDAP. Team-manager appointment and CCB approval authority remain explicit administrative decisions. Only active users granted `CCB approver` authority participate in new CCB approval gates. Normal and emergency changes follow manager assessment and CCB authorization.

Operational ownership is separate from approval authority. Every incident, service request, and change has one owning IT fulfillment team. An active member of that team, its configured manager, or a platform administrator can change operational fields. Assignees must remain active members of the owning team. Active members and managers of every IT fulfillment team can read all incidents and changes through lists, search, direct URLs, dashboards, analytics, task boards, history and attachments. This shared visibility supports triage, handoffs and operational awareness, but only the owning team or a platform administrator can change the parent ticket. A named approver can see the record needed for the decision, but CCB membership grants approval authority only and does not grant cross-team operational control. Administrator actions remain audited.

3. End-user guide

REST API documentation

The complete integration guide is maintained in `docs/API_REFERENCE.md` and is served by the running application at `/api/v1/docs`. The machine-readable OpenAPI 3.1 contract is `/api/v1/openapi.json`. It documents authentication, scopes, tenant and team authorization, field projections, idempotency, pagination, ticket operations, workflow triggers, monitoring ingestion, errors, retries, cURL and Python examples, and the explicit current compatibility boundary.

Governed record projections

Record visibility and field visibility are separate controls. Central tenant-aware object policies decide whether a user can discover or access a record. The validated Git-backed policy in `config/field_projections.json` then decides which fields may leave the application for that resource and audience.

The registry covers every supported record REST response, signed audit export, JSON search result, monitoring acknowledgement, workflow acknowledgement and interactive mutation acknowledgement. Unknown resources, unknown audiences, duplicate fields and fields outside a resource allowlist fail application startup. Requester ticket projections exclude internal assignment details; internal projections are limited to agent, manager and administrator audiences. Adding a new record API or export requires a reviewed registry binding and adversarial projection test.

Browser request security

Every state-changing browser request requires a session-bound CSRF token. ServiceOps inserts tokens into all rendered POST forms and supplies the token to interactive workspace actions through a dedicated request header. Tokens rotate after authentication. Missing, expired or mismatched tokens fail closed with HTTP 400 and make no data change.

Session cookies are HttpOnly and SameSite=Lax, and `SESSION_COOKIE_SECURE` defaults to `true` everywhere (Docker Compose, Helm, and the RPM package), not only in a production Helm profile. Outside test mode, the application refuses to start with insecure cookies unless `ALLOW_INSECURE_SESSION_COOKIES=true` is also set explicitly; that escape hatch exists only for an isolated plain-HTTP localhost/development deployment and must never be set in production. `SESSION_LIFETIME_MINUTES` controls the bounded session lifetime.

A request whose authenticated user has no resolvable `tenant_id` is logged out and rejected with HTTP 403 rather than silently defaulted to a tenant -- tenant resolution fails closed.

A Content-Security-Policy header (`default-src 'self'`, restricting script, style, image, font, and connection sources to the same origin) is sent on every response.

Database migration gate

Production schema changes are controlled by Alembic revisions. Compose runs the migration gate before initialization. Kubernetes uses a dedicated Helm pre-install/pre-upgrade migration Job; normal application replicas set `AUTO_MIGRATE=false` and refuse to start unless the database is already at the required revision.

Revision `20260726_0001` adopts a complete existing ServiceOps schema without rewriting operational records, or creates the schema for a fresh installation. Its downgrade is intentionally non-destructive: removal of an adopted baseline requires restoring the validated pre-migration backup. Every later schema revision must provide and test its own one-revision downgrade.

Revision `20260726_0002` creates the installation's default tenant and assigns all existing tenant-owned root records to it without deleting or recreating operational data. The application constrains ticket, request, enterprise, knowledge, asset, catalog, CMDB, notification, administration, approval, and global-search roots to the signed-in user's tenant. Cross-tenant direct identifiers return HTTP 404. Automated migration evidence covers fresh installation, existing-schema adoption, one-revision downgrade, roll-forward, row preservation, and tenant backfill. A deployment still requires a validated backup and an environment-specific rehearsal before production promotion.

For bundled PostgreSQL, run the guarded rehearsal before promoting an upgrade:

```
./serviceops rehearse-migrations 100000
```

The command clones the installed database into a database whose name must end in `_migration_rehearsal`, adds the requested number of isolated representative records, performs a one-revision downgrade and roll-forward, and

compares table counts plus stable identity/content fingerprints. A cleanup trap removes the clone on success or failure. The command refuses external-database mode because those rehearsals must use an isolated database supplied by that provider.

Revision `20260726_0003` extends every audit event with a UUID, request correlation identifier, source context, integrity algorithm, previous-event hash, and event hash. Existing events are deterministically sealed into a tenant-specific legacy SHA-256 chain; new events use HMAC-SHA-256 with the deployment integrity key. PostgreSQL and SQLite triggers reject audit UPDATE and DELETE operations at the database boundary. The administrator audit page verifies the entire tenant chain before displaying its status. Audit export is blocked if verification fails and otherwise returns ordered JSON evidence with a detached signature in `X-ServiceOps-Audit-Signature`.

Revision `20260726_0011` gives every HMAC event a signing-key identifier. Administrators can rotate forward from the audit page only after full-chain verification succeeds. Historical secrets remain encrypted for verification; neither events nor hashes are rewritten. The export response identifies its signing key in `X-ServiceOps-Audit-Key-ID`. `AUDIT_INTEGRITY_KEY_FILE` supports an initially mounted secret and takes precedence over the environment value.

The same page governs a minimum seven-year retention period, legal hold, and the requirement for external immutable evidence. Primary audit rows are never automatically purged. When audit streaming is enabled, each new audit event is committed transactionally to the durable outbox. Only active `siem` connections receive `audit.created` events; ordinary webhooks and Teams connections cannot receive this security stream. Delivery uses HTTPS, event identifiers, timestamps and HMAC signatures, with bounded retry and delivery evidence. The organization must still validate its actual immutable SIEM/WORM destination and retention controls.

Protect `AUDIT_INTEGRITY_KEY` as a rotated secret independent from database operator access. When it is absent, ServiceOps uses the settings encryption key and finally the application secret. Changing the effective key without a governed chain rollover makes prior HMAC events unverifiable. Database append-only enforcement does not replace external immutable retention; export signed evidence to the organization's governed WORM or SIEM destination.

Declarative action and field authorization

`config/authorization.json` is the Git-controlled source for the initial role/action vocabulary. It declares discover, read, public/internal comment, create, update, assign, accept, transition, resolve, close, reopen, approve, delegate, relate, export, report, delete, purge, configure, administer, and security-administration actions. Startup fails if the policy is malformed or a role contains an unknown action. Browser handlers and future REST handlers use the same `serviceops_core.security` interface.

Object visibility remains relationship-aware and tenant-aware. Field projection is a separate decision: requesters viewing their own incident receive its public description, public comments, public attachments, state, priority, category, assignment summary, and ownership summary. They do not receive internal ticket history, work notes, change risk/plans, approval records, SLA internals, major incident coordination, affected-CI internals, operational tasks, or checklists. Active fulfillers retain the internal projection according to their object and team authorization. Client-supplied fields never expand that projection.

REST API v1

Administrators create and revoke REST identities under **Administration → API clients**. Each client is bound to one active user in the same tenant and inherits that user's object, relationship, field, and action authorization. Explicit scopes further restrict the client. Plaintext bearer tokens are shown only in the creation response; ServiceOps stores an HMAC hash and prefix, never the recoverable token. Protect the independently generated `API_TOKEN_PEPPER`; changing it revokes every existing token by design.

The initial `/api/v1` contract provides access-aware cursor-paginated ticket listing, individual ticket retrieval, incident creation, controlled ticket updates, and an OpenAPI 3.1 discovery document. Unsafe operations require an `Idempotency-Key`. Repeating identical content replays the stored JSON result; reusing a key with different content fails with HTTP 409. API errors include the correlated request identifier. Writes enter the append-only audit chain with the acting user and API client identifier.

Responsive PWA

ServiceOps publishes a dynamic company/instance manifest and root-scoped service worker. The worker caches only CSS and JavaScript shell assets. It does not intercept or cache HTML, authentication, API responses, tickets, requests, attachments, or other operational records. Installation outside localhost requires HTTPS. Offline operational records remain explicitly deferred until encrypted device storage, remote revocation, and data-loss policy are approved.

Durable integrations and monitoring

Revision `20260726_0005` adds a PostgreSQL-backed outbox, per-channel delivery evidence, encrypted webhook/Teams connections, authenticated monitoring sources, and deduplicated monitoring events. Application transactions create notifications and outbox events together. The separate Compose/Kubernetes worker claims due events with `FOR UPDATE SKIP LOCKED`, delivers them, records each attempt, retries with bounded exponential delay, and moves an event to `Dead` after five failed processing attempts. A successful channel is not sent again during retries of another channel.

SMTP is configured post-installation under platform settings. STARTTLS is enabled by default; hostname, port, account, encrypted password, and from address remain administrator-controlled. Signed webhooks use HTTPS and carry event ID, timestamp, and `HMAC-SHA-256` signature headers. Teams connections use the same durable worker with Teams-compatible message payloads. Literal loopback, private, link-local, and non-HTTPS webhook targets are rejected at configuration time. At delivery time the application re-resolves the destination hostname and rejects it if it now resolves to a non-global address, disables automatic redirect-following, and manually re-validates and re-resolves the target on every redirect hop (up to three), closing most of the DNS-rebinding/redirect-SSRF gap without relying solely on network egress controls. A narrow TOCTOU window remains between that re-resolution and the HTTP client's own connection-time DNS lookup; true IP pinning or a controlled outbound proxy is tracked as future work. Defense-in-depth egress firewall/DNS controls at the network layer are still recommended in production.

Administrators create monitoring sources under **Integrations** and bind each source to an active IT fulfillment team. The source token is displayed once and stored only as a hash. `POST /api/v1/monitoring/{source_id}/events` requires that bearer token, validates severity and payload limits, deduplicates by source/external ID, creates an EVT record, assigns an EVTASK investigation to the configured team, maps critical/high/medium/low severity to P1/P2/P3/P4, and records ingestion in the append-only audit chain.

Sign in

The sign-in page offers the identity methods enabled by the administrator:

Workflow state integrity

Approval-derived states are controlled by the approval engine and cannot be selected manually. A change remains `Awaiting Approval` until its active manager and CCB gates complete; only then can it move from `Approved` to `In Progress`. The ticket form and visual task board use the same server-side transition policy. Administrators cannot cast another named approver's vote. After approval, the owning team or a platform administrator can start, resolve, reopen, or close the work. A member of SSD can read a Unix-owned ticket for operational awareness but cannot operate it merely because that person is a manager or CCB approver.

Enterprise records with requested approvals remain locked in `Awaiting Approval`. Catalog fulfillment tasks cannot begin until their RITM approval chain completes, and terminal records cannot be reopened through a generic update endpoint. Invalid transitions return HTTP 409 and make no data change.

ITIL related-record model

ServiceOps treats operational work as a governed network rather than a ticket chain:

Record · Purpose and supported relationships

INC · Restore service; parent/child INC, primary and affected CIs, PRB, fix CHG, caused-by CHG, converted REQ

PRB · Root cause, workaround, known error and permanent fix; many INCs, PTASKs, multiple CHGs and knowledge

PTASK · Independently assigned investigation or resolution work package under a PRB

CHG · Risk, authorization, schedule and overall implementation control; related INCs, PRBs, RITMs, CIs and services

CTASK · Planning, implementation, testing or review work assigned to a specific team under a CHG

REQ · User order container containing one or more RITMs

RITM · Catalog item, variables, approvals and fulfillment stage; may link to a controlled CHG

SCTASK · Independently assigned fulfillment work under a RITM, in parallel or after a predecessor

CTASK, PTASK and SCTASK are distinct records. A required open CTASK prevents its parent CHG from completing. A required open PTASK prevents its PRB from completing. A RITM completes only after all SCTASK work reaches a terminal result, and a REQ completes only after all its RITMs complete.

Major-incident coordination remains an extension of an INC. It records proposed or accepted major status, business impact, coordinator and communications; it does not create a separate incident prefix. If the administrator enables parent incident state synchronization, supported state changes propagate to linked child INCs and are recorded in each child history.

Ticket history and approval-safe change revisions

Every operational record page contains its own chronological history. The history records the actor, precise time, event, field, previous value, new value and supporting details for record creation, state/priority/assignment changes, comments, attachments, checklist actions, related records, CI links, PTASK/CTASK/SCTASK work, and approval decisions.

For a CHG, the following are material approval inputs: purpose and description, change type, risk, impact, implementation plan, test plan, backout plan, planned window and primary CI. Editing any of them:

1. preserves the previous chain and votes as historical evidence;
2. marks the previous approval chain `Superseded`;
3. increments the CHG plan revision;
4. returns the CHG to `Awaiting Approval`;
5. creates a new manager/CCB approval chain; and
6. sends an in-application reapproval notification to every configured

approver.

An old approval is therefore never silently applied to a materially different change plan.

Disposable full-function test fixture

`tools/load_test_fixture.py` is an explicit non-production utility and is never executed during normal startup. It must be used only after an intentional database reset. The fixture displays a permanent red warning banner and creates an administrator plus one manager/CCB approver for every IT team:

Team · Username · Password

Administration · `admin` · `admin`

CoreApps · `coreapps` · `coreapps`

Database · `db` · `db`

Network · `network` · `network`

Windows · `windows` · `windows`

Unix · `unix` · `unix`

SSD · `ssd` · `ssd`

It also creates disposable catalog items, a configuration item, an asset, a knowledge article, a service offering, AD mappings, team-manager assignments, Service Desk memberships, and CCB approval authority. Never expose an instance containing this fixture to a network or reuse it for production data.

`tools/load_demo_dataset.py` (added in 1.27.18) is a second, non-production loader with the same `--confirm-non-production` requirement and identical production-image exclusion. Instead of a minimal fixture, it builds a realistic, deep dataset for manual exploration and testing: ~19 configuration items across three business-service dependency trees (application → database → server, plus shared network and storage CIs) with ~32 CI relationships, two named users per IT team (manager + agent; managers are also added as CCB approvers), five plain requester accounts, and a representative spread of 8 incidents, 3 problems (each with problem tasks), 3 changes (Standard/ Normal/Emergency, each with change tasks), 2 catalog requests, and 5 knowledge articles. Every seeded user's password equals its username. Seeded changes and problems set their `state` directly rather than driving the live approval engine, so they render immediately without a pending approval blocking the screen — submit a *new* change or request through the UI against this data to exercise the live approval workflow itself. Keycloak SSO, AD/LDAP, and local administrator. The local administrator is a break-glass account and should not be used for routine work.

Navigation

The left application navigator contains role-appropriate modules. The top bar contains global search, favorites, recent history, notifications, help, and preferences. The categorized Preferences workspace controls density, font scale, high contrast, reduced motion, accessible tooltips, keyboard shortcuts, date/list presentation, whether the sidebar stays pinned open, and the start page. The user name in the navigation footer opens the self-service profile, where a user can maintain their name, email, title, phone, time zone and date format. Role, active state, department, team membership, manager authority and CCB authority are not self-service fields. ServiceOps is light-only; there is no theme selector (ADR-012).

Administrators use Administration home for a capability-oriented entry point to identity, ITIL configuration, workflow, CMDB, integrations, diagnostics and analytics. Users & roles provides tenant-scoped search and an editable user record. Only administrators with `security_administer` may alter role, active state or department. AD-sourced team membership continues to reconcile from configured directory mappings and is not replaced by profile editing.

Incidents

Create an incident with a concise title, observable symptoms, business impact, category, and urgency. Agents classify, prioritize, assign, comment, attach evidence, execute checklists, observe SLA timers, resolve, and close it.

The incident record uses a dense, two-column operational form. Its header keeps Update and permitted Resolve actions visible, while the body exposes the number, caller, contact type, state, category/subcategory, impact/urgency, calculated priority, service offering, primary configuration item, assignment group, assignee, notification preference, short description, and description. The immutable Event history follows the form directly and records field-level old/new values with actor and timestamp. Authorization remains server-enforced: the owning team controls operational mutations even though active IT teams can read incidents.

The same task-derived interaction model is used for changes, problems and other enterprise operational records, and request containers. Every supported task record keeps its number/type identity and permitted actions in a consistent header, presents common state/priority/requester/assignment fields in the same two-column pattern, places field-level Event history immediately below the record, and then exposes type-specific governance, tasks, approvals, SLAs, attachments, work notes and related records. List workspaces use a consistent New/search/filter bar, encoded-filter summary, record count, pagination, priority indicators and operational columns.

Requests and catalog

Select a catalog item, supply its variables and business justification, and submit. ServiceOps creates REQ, RITM, approvals, and SCTASK records. Follow the request page for approval and fulfillment status.

Each catalog item has an administrator-controlled default fulfillment route under **ITIL configuration → Catalog item fulfillment routing**. The generated initial SCTASK inherits that team. Laptop and Software catalog items are routed to

Windows by default. Administrators can route any current or future catalog item to another active fulfillment team without changing application code. The same screen creates and edits catalog items, including name, category, description, delivery target, approval requirement, active availability and fulfillment team. New items require an explicit active fulfillment team. Legacy items without an explicit route use the active Service Desk as a controlled fallback; ordering fails closed if neither an explicit team nor Service Desk is active. Inactive items cannot be ordered through either the UI or a direct endpoint.

Catalog request visibility is need-to-know. A REQ and its RITMs are visible only to the requested-by/requested-for users, members or managers of the configured fulfillment team, teams assigned an SCTASK, specifically named approval voters, and platform administrators. The same policy protects the request list, direct URL access, dashboard counts and global search. Membership in an unrelated team, such as Unix, does not expose a Windows-routed request. Administrators retain complete visibility for platform oversight and audit.

Changes

Every change must state type, affected CI, owning team, risk, impact, planned window, implementation plan, test plan, and executable backout plan. Review conflicts before approval. CCB approval is authorization, not a substitute for technical validation or membership in the owning implementation team.

Knowledge, CMDB, assets, and boards

Search knowledge before opening duplicate work. Use the CMDB relationship view to understand service impact. Asset pages track accountable inventory. The visual task board changes the underlying ticket state and therefore remains audited and role controlled.

Release 1.27.18 enriches the Configuration Item record beyond the original name/class/environment/status/IP/owner fields: each CI now also carries a description, lifecycle state (Planned/In Use/Maintenance/Retired/Disposed, distinct from operational status), business criticality (Critical/High/ Medium/Low), serial number, vendor, model, physical/logical location, cost center, discovery source (Manual/Discovery scan/Import/API), install and warranty-expiry dates, an owning support group (in addition to the individual owner), and a free-form JSON attributes bag for anything not covered by a named column. CI-to-CI relationships are now tenant-scoped in their own right (`ci_relationship.tenant_id`) rather than relying solely on their parent/child CI's tenant, and a given relationship type between the same two CIs can no longer be created twice. Administrators manage all of this from `/cmdb` and `/cmdb/<id>/edit`; the REST CMDB auto-registration endpoint (`PUT /api/v1/cmdb/configuration-items`, `cmdb:write` scope, see `API_REFERENCE.md` §9) has **not** yet been extended to accept these new fields — it remains upsert-by-name over the original five fields only, so richer CI data must currently be entered through the web UI. Migration: `20260729_0023_cmdb_enrichment.py`.

4. Deployment decision

Environment · Application · Database · Upload storage

Single server · Docker Compose · Bundled PostgreSQL · Docker volume

Single production server · Docker Compose behind HTTPS proxy · Bundled or external PostgreSQL · Docker volume with backups

Enterprise production · Kubernetes/Helm, 3+ replicas · Managed HA PostgreSQL · RWX CSI volume or object-storage extension

Production Kubernetes must use an externally operated, highly available PostgreSQL service.

5. Docker installation

Run `./serviceops install web`, open `http://127.0.0.1:8090`, configure the profile, database, identity providers, and listener, validate all checks, confirm, and deploy. Keep the terminal open until the browser reports a healthy deployment.

Use `./serviceops status`, `health`, `logs`, `backup`, `restore`, and `doctor` for lifecycle operations. Put Caddy, NGINX, or an enterprise load balancer in front of the loopback-bound application and terminate TLS there.

6. Kubernetes prerequisites

- Kubernetes 1.27 or newer with at least three worker nodes for HA.
- Helm 3, kubectl, a default-deny-capable CNI, and a CSI storage provider.
- A private image registry and immutable ServiceOps image tag or digest.
- External HA PostgreSQL with TLS, automated backups, and point-in-time recovery.
- RWX upload storage when application replicas exceed one.
- An ingress controller, DNS, and automated TLS certificate management.
- Metrics Server for HPA; cluster monitoring and log aggregation.
- A secrets controller or external vault for steady-state secret rotation.

7. Kubernetes installation

1. Build, scan, sign, and push an immutable image.
2. Copy `deploy/kubernetes/values-production.example.yaml` to `deploy/kubernetes/values-production.yaml`.
3. Set registry, immutable tag, ingress, storage class, replicas, identity endpoints, and role mappings.
4. Run `./serviceops install kubernetes --preflight`.
5. Run `./serviceops install kubernetes`.
6. Confirm rollout, Helm test, ingress TLS, login, record creation, upload, approval, notification, and audit behavior.

The installer labels the namespace for Restricted Pod Security, creates secrets from a protected temporary file, uses `helm upgrade --install --atomic --wait`, waits for rollout, and runs the packaged `/ready` test.

8. Kubernetes chart controls

- Non-root UID/GID, RuntimeDefault seccomp, all capabilities dropped.
- Read-only root filesystem and disabled service-account-token mounts.
- Startup, readiness, and liveness probes with distinct semantics.
- Rolling update with zero unavailable replicas.
- Topology spreading across zones and hosts.
- PodDisruptionBudget and optional HPA.
- Resource requests and limits.
- NetworkPolicy for ingress and required egress ports.
- Persistent upload claim and optional ingress/TLS.
- JSON schema validation and a Helm test hook.

Tune topology keys to the labels present in the target cluster. A PDB protects only against voluntary disruptions; it does not protect against node, zone, or application failure.

9. Identity configuration

Local administrator

Keep local authentication enabled for one vaulted break-glass administrator. Test it quarterly under an approved access procedure. Rotate after every use.

AD/LDAP

Use LDAPS or StartTLS, certificate validation, a least-privilege read-only bind identity, a narrow base DN, an escaped username filter, and explicit group-role mappings. Validate bind, search, user bind, disabled-user behavior, group mapping, certificate expiry, and directory outage behavior.

Keycloak

Use a confidential OIDC client, authorization-code flow, exact HTTPS redirect URI, short-lived tokens, approved realm-role claims, client-secret rotation, and restrictive redirect/web-origin settings. Validate login, logout, expired session, revoked user, missing email, role changes, and provider outage.

10. Post-deployment system settings

Administrators can open **Administration → System settings** after deployment to change the platform name, company name, PNG logo, primary and accent colors, support identity, display defaults, LDAP, Keycloak, encrypted provider secrets, security limits, workflow defaults, and notification identity.

Each field is marked either **Live** or **Restart required**. Live settings are read from PostgreSQL by every request. Restart-required identity-client changes must be followed by a complete Compose restart or Kubernetes rollout so every application instance receives the same client configuration.

Database topology, replicas, storage, ingress, and TLS remain controlled by Compose or Helm and are displayed read-only. This prevents one pod from silently diverging from the declared infrastructure.

Sensitive values are encrypted before storage and are never returned to the browser. Set a durable `SETTINGS_ENCRYPTION_KEY` during installation; changing or losing that key makes existing encrypted settings unreadable.

Priority and SLA policy

Ticket priority is calculated from impact and urgency using the validated, Git-backed `config/priority_matrix.json`. Agents cannot silently override the result. A manager or administrator may select a different priority only while supplying an auditable reason of at least ten characters.

Administrators manage business calendars and SLA definitions under **ITIL configuration**. Calendars use IANA timezone names, explicit business weekdays and opening hours, plus named excluded holiday dates. An SLA without a calendar remains a 24x7 wall-clock commitment. A calendar change applies to future SLA attachments; it does not silently rewrite targets already in flight.

The worker detects newly breached commitments, writes one breach event to the SLA evidence stream, records the breach in the ticket history, and notifies the owning-team manager and assigned engineer through the durable outbox. Monitor the worker continuously; a stopped worker delays escalation but does not lose the stored target or breach state.

Declarative workflow operations

Workflow source is controlled in `config/workflows.json`. On startup ServiceOps validates the package and publishes a new immutable PostgreSQL runtime version only when the canonical specification changes. Administrators can inspect the deployed hash and versions, redeploy the packaged source, and run a mutation-free simulation under **Administration → Workflows**.

The supported foundation accepts `ticket.state_entry` events, equality, inequality, membership and empty-value conditions over explicitly allowed ticket fields, plus three actions: add ticket history, notify the requester, and notify the owning-team manager. Arbitrary administrator scripts and unsupported fields/actions fail package validation.

State transitions enqueue a unique correlated job in the same transaction as the operational change. The worker claims jobs using PostgreSQL coordination, records the exact published version and masked structured input/output, and commits actions atomically. Failures use bounded exponential retry and enter `Dead` after five attempts. Monitor both workflow jobs and execution evidence from the administration page; investigate dead jobs before replaying them.

Release 1.13.0 extends the runtime with durable waits, per-action execution evidence, manual/API/SLA-breach triggers, per-workflow rate limits, controlled dead-job replay, and safe terminal compensation. A wait commits its cursor and resume timestamp to PostgreSQL; after restart the worker continues at the next action and does not repeat completed steps.

API workflow triggers require the explicit `workflows:execute` scope, the acting user's operational permission for the ticket, and an idempotency key. Rate-limited jobs are delayed without consuming a failure attempt. Failed jobs retry five times; compensation runs only after the terminal failure and is currently restricted to an auditable ticket-history action because delivered notifications cannot truthfully be undone.

Release 1.13.0 does not yet claim scheduled recurrence, reusable subflows, general rollback, broad record mutation, concurrency quotas, or complete multi-environment configuration promotion. Those remain governed backlog work.

Release 1.14.0 adds reusable subflows to workflow package schema v2. Subflow references are validated before deployment, unknown dependencies fail closed, and dependency cycles are rejected. Valid subflows are expanded into the immutable published workflow specification, so runtime execution never depends on mutable external fragments.

Administrators configure recurring ticket schedules under **Administration → Workflows**. Each schedule is tenant-scoped, targets one ticket, uses a bounded minute interval, and can be enabled or disabled without deleting its evidence. Concurrent workers claim due schedules with PostgreSQL row locking. The event and next-run advancement commit together. If the system was offline for several intervals, it emits one event and advances directly to the next future interval instead of creating a catch-up storm.

Calendar expressions, blackout windows, package dependencies, per-tenant concurrency quotas and production-scale scheduler failure testing remain governed backlog work.

Bootstrap credential retirement

ServiceOps reads `ADMIN_PASSWORD_FILE` before the legacy `ADMIN_PASSWORD` environment value. Kubernetes separates the bootstrap credential from runtime secrets and never injects it into workers. Compose workers likewise receive no bootstrap administrator credential.

After the first successful installation:

1. Sign in as the local administrator.
2. Use **Administration → Change password** to rotate the initial credential

into a unique vault-managed password. This increments the account authentication version and invalidates every other browser session.

3. Store the new credential in the organizational emergency-access vault.
4. Run `./serviceops retire-bootstrap-secret`, verify the active administrator,

type the explicit confirmation, and wait for the recreated containers to pass health checks.

5. Run `./serviceops doctor`; the worker-secret isolation check must pass.

The retirement operation clears only bootstrap injection from `.env`; it does not erase or reset the administrator's salted password hash in PostgreSQL.

Release evidence

Tagged releases run `.github/workflows/supply-chain.yml`. Every referenced GitHub Action is pinned to a full commit SHA. The gate runs the complete test suite, builds with maximum provenance, blocks fixable HIGH and CRITICAL operating-system or library vulnerabilities, generates a CycloneDX image SBOM, and publishes only after successful validation. The resulting digest is signed keylessly with GitHub OIDC and receives GitHub SLSA build-provenance and SBOM attestations. Registry digest inspection and `gh attestation verify` must both succeed.

Kubernetes values must provide `image.digest`; application, worker and migration workloads use `repository@sha256:digest`, never a mutable tag. The installer requires `SERVICEOPS_GITHUB_ORGANIZATION`, installs the pinned Sigstore Policy Controller and GitHub trust policy, and labels the ServiceOps namespace for enforcement. Unsigned or untrusted matching organization images are rejected at admission. Run `python tools/verify_supply_chain.py` locally to detect release-policy drift.

Runtime Python and bundled PostgreSQL base images are pinned by digest and Python dependencies use exact versions. Generate the release record with:

```
python tools/release_evidence.py \
--version 1.26.3 \
--image serviceops-app:1.26.3 \
--output release-evidence/serviceops-1.26.3.json
```

The output records Git state, SHA-256 hashes for non-ignored source files, a combined manifest hash, image reference and CycloneDX component inventory. A dirty-tree marker is evidence, not approval. External vulnerability scanning, image signing, provenance publication, registry verification and admission enforcement remain required before organizational production approval.

11. Security operations

- Enforce TLS externally and enable HSTS only after HTTPS is proven.
- Store secrets in a vault; never commit `.env`, values secrets, or kubeconfig.
- Restrict namespace RBAC and database privileges.
- Scan source, dependencies, image, manifests, and exposed endpoints in CI.
- Sign images and enforce admission verification where supported.
- Forward application, ingress, Kubernetes audit, database, and identity logs.
- Alert on repeated login failures, admin changes, approval anomalies, SLA

breaches, crash loops, readiness loss, saturation, and storage pressure.

- Patch base images and dependencies under change control.

12. Backup and recovery

Back up PostgreSQL and uploads as one recovery set. Encrypt backups, store them outside the cluster, apply retention and immutability, and record restore test evidence. For managed PostgreSQL enable PITR. Snapshotting a running database volume alone is not a proven logical backup.

Quarterly recovery test:

1. Run `./serviceops rehearse-recovery` and `./serviceops rehearse-pitr`; retain

both evidence records.

2. Restore into an isolated environment.

3. Restore database to the selected recovery point.
4. Restore uploads from the matching recovery set.
5. Deploy the matching immutable application version.
6. Validate counts, attachments, identities, approvals, audit, and critical

workflows.

7. Record actual RPO/RTO and corrective actions.

The logical recovery command deliberately reports `pitr_proven=false`: `pg_dump` cannot be replayed with WAL. The separate PITR rehearsal uses `pg_basebackup`, continuous WAL archiving, and a named recovery target on disposable clusters, and reports `pitr_proven=true` only after boundary and ServiceOps integrity checks pass.

13. Upgrades and rollback

Run `./serviceops rehearse-upgrade` first. It creates a verified rollback set and validates the candidate image and migration against an isolated clone while the source remains healthy. Back up first. Review release notes and schema compatibility. Render and lint the Helm chart, deploy to staging, run workflow regression and load tests, then use `helm upgrade --install --atomic --wait`. Observe error rate, latency, readiness, database health, and queues. `--atomic` rolls Kubernetes resources back when the upgrade fails, but database schema/data rollback still requires a release-specific tested procedure.

14. Monitoring and SLOs

Monitor availability, request rate, latency percentiles, HTTP errors, worker saturation, pod restarts, readiness, CPU/memory throttling, database connection usage, query latency, replication/backup health, volume capacity, login errors, notification failures, and SLA breach rate. Define business-approved SLOs and page only on actionable symptoms.

15. Incident response

Declare severity and incident commander, preserve evidence, stabilize service, communicate on a fixed cadence, use tested rollback/failover, validate recovery, and create a blameless problem record with corrective actions. Never delete audit or operational evidence during response.

16. Production acceptance checklist

- ■ Immutable, scanned image from trusted registry
- ■ External HA PostgreSQL with TLS, PITR, and restore test
- ■ Three or more application replicas across failure domains
- ■ RWX persistent uploads and tested restore
- ■ TLS ingress, DNS, HSTS, security headers
- ■ Restricted Pod Security and least-privilege RBAC
- ■ Network policies validated with the actual CNI and endpoint topology
- ■ LDAP/Keycloak end-to-end tests and vaulted break-glass account
- ■ Monitoring, logs, alert routing, dashboards, and runbooks
- ■ Load, soak, failover, node-drain, rollback, and disaster-recovery tests
- ■ Security review, penetration test, and formal go-live approval